Code :

```
#include <iostream>
using namespace std;
// Helper function to check existence
bool exists(int arr[], int size, int val) {
for (int i = 0; i < size; i++)
if (arr[i] == val)
return true;
return false;
}
// Union
void unionf(int a[], int b[], int m, int n) {
int c[100], k = 0;
for (int i = 0; i < m; i++)
c[k++] = a[i];
for (int i = 0; i < n; i++)
if (!exists(a, m, b[i]))
c[k++] = b[i];
cout << "\nUnion (A ∪ B) = { ";
for (int i = 0; i < k; i++)
cout << c[i] << " ";
cout << "}";
}
// Intersection
void inter(int a[], int b[], int m, int n) {
cout << "\nIntersection (A ∩ B) = { ";
for (int i = 0; i < n; i++)
if (exists(a, m, b[i]))
cout << b[i] << " ";
cout << "}";
```

```
}
// Difference
void dif(int a[], int b[], int m, int n) {
cout << "\nDifference (A − B) = { ";
for (int i = 0; i < m; i++)
if (!exists(b, n, a[i]))
cout << a[i] << " ";
cout << "}";
cout << "\nDifference (B − A) = { ";
for (int i = 0; i < n; i++)
if (!exists(a, m, b[i]))
cout << b[i] << " ";
cout << "}";
}
// Symmetric Difference
void sym(int a[], int b[], int m, int n) {
cout << "\nSymmetric Difference (A ⊕ B) = { ";
for (int i = 0; i < m; i++)
if (!exists(b, n, a[i]))
cout << a[i] << " ";
for (int i = 0; i < n; i++)
if (!exists(a, m, b[i]))
cout << b[i] << " ";
cout << "}";
}
int main() {
int m, n;
int a[100], b[100];
cout << "Enter number of elements in set A: ";
cin >> m;
```

```
cout << "Enter elements of set A: ";
for (int i = 0; i < m; i++)
cin >> a[i];
cout << "Enter number of elements in set B: ";
cin >> n;
cout << "Enter elements of set B: ";
for (int i = 0; i < n; i++)
cin >> b[i];
unionf(a, b, m, n);
inter(a, b, m, n);
dif(a, b, m, n);
sym(a, b, m, n);
return 0;
}
```

iii) Difference Operation (A − B)

• Traverse Set A.

• Print elements that are not found in Set B.

iv) Symmetric Difference

• Combine the difference of A − B and B − A.

• Displays elements unique to each set.

```
#include <iostream>
#include <cmath>
using namespace std;

class Polynomial {
   int coeff[100];
   int power[100];
   int terms;

public:
   // Constructor
   Polynomial() {
      terms = 0;
   }

   // Insert term
   void insertTerm(int c, int p) {
      coeff[terms] = c;
      power[terms] = p;
      terms++;
   }

   // Create polynomial
   void createPolynomial() {
      int n, c, p;

      cout << "Enter number of terms: ";
      cin >> n;

      for(int i = 0; i < n; i++) {
         cout << "Enter coefficient and power: ";
```

```
        cin >> c >> p;

        insertTerm(c, p);
    }
}

// Display polynomial
void display() {
    for(int i = 0; i < terms; i++) {
        cout << coeff[i] << "x^" << power[i];

        if(i != terms - 1)
            cout << " + ";
    }
    cout << endl;
}

// Add two polynomials
Polynomial add(Polynomial p2) {
    Polynomial result;

    int i = 0, j = 0;

    while(i < terms && j < p2.terms) {

        if(power[i] == p2.power[j]) {
            result.insertTerm(coeff[i] + p2.coeff[j], power[i]);
            i++;
            j++;
        }
```

```
        else if(power[i] > p2.power[j]) {
            result.insertTerm(coeff[i], power[i]);
            i++;
        }

        else {
            result.insertTerm(p2.coeff[j], p2.power[j]);
            j++;
        }
    }

    // Remaining terms
    while(i < terms) {
        result.insertTerm(coeff[i], power[i]);
        i++;
    }

    while(j < p2.terms) {
        result.insertTerm(p2.coeff[j], p2.power[j]);
        j++;
    }

    return result;
}

// Evaluate polynomial
int evaluate(int x) {
    int sum = 0;

    for(int i = 0; i < terms; i++) {
        sum += coeff[i] * pow(x, power[i]);
```

```
        }

        return sum;
    }
};

int main() {

    Polynomial p1, p2, p3;

    cout << "Enter First Polynomial" << endl;
    p1.createPolynomial();

    cout << "\nEnter Second Polynomial" << endl;
    p2.createPolynomial();

    cout << "\nFirst Polynomial: ";
    p1.display();

    cout << "Second Polynomial: ";
    p2.display();

    // Addition
    p3 = p1.add(p2);

    cout << "\nAddition of Polynomials: ";
    p3.display();

    // Evaluation
    int x;
    cout << "\nEnter value of x: ";
```

```
    cin >> x;

    cout << "Value of First Polynomial = " << p1.evaluate(x) << endl;

    return 0;
}
```

```
#include <iostream>
using namespace std;

// Node structure
struct Node {
    int id;
    string name;

    Node* prev;
    Node* next;
};

// Doubly Linked List Database
class Database {

    Node* head;
    Node* tail;

public:

    Database() {
        head = NULL;
        tail = NULL;
    }

    // Insert record
    void insertRecord(int id, string name) {

        Node* newNode = new Node();

        newNode->id = id;
```

```
    newNode->name = name;

    newNode->prev = NULL;
    newNode->next = NULL;

    // If list empty
    if(head == NULL) {
        head = tail = newNode;
    }
    else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }

    cout << "Record Inserted\n";
}

// Delete record
void deleteRecord(int id) {

    Node* temp = head;

    while(temp != NULL) {

        if(temp->id == id) {

            // First node
            if(temp == head) {
                head = head->next;
```

```
            if(head != NULL)
                head->prev = NULL;
        }

        // Last node
        else if(temp == tail) {
            tail = tail->prev;
            tail->next = NULL;
        }

        // Middle node
        else {
            temp->prev->next = temp->next;
            temp->next->prev = temp->prev;
        }

        delete temp;

        cout << "Record Deleted\n";
        return;
      }

      temp = temp->next;
   }

   cout << "Record Not Found\n";
}

// Modify record
void modifyRecord(int id) {
```

```
    Node* temp = head;

    while(temp != NULL) {

        if(temp->id == id) {

            cout << "Enter new name: ";
            cin >> temp->name;

            cout << "Record Modified\n";
            return;
        }

        temp = temp->next;
    }

    cout << "Record Not Found\n";
}

// Display forward
void displayForward() {

    Node* temp = head;

    cout << "\nRecords Forward:\n";

    while(temp != NULL) {

        cout << "ID: " << temp->id
             << " Name: " << temp->name << endl;
```

```
            temp = temp->next;
        }
    }

    // Display backward
    void displayBackward() {

        Node* temp = tail;

        cout << "\nRecords Backward:\n";

        while(temp != NULL) {

            cout << "ID: " << temp->id
                << " Name: " << temp->name << endl;

            temp = temp->prev;
        }
    }
};

int main() {

    Database db;

    int choice, id;
    string name;

    do {

        cout << "\n----- DATABASE MENU -----\n";
```

```
cout << "1. Insert Record\n";
cout << "2. Delete Record\n";
cout << "3. Modify Record\n";
cout << "4. Display Forward\n";
cout << "5. Display Backward\n";
cout << "6. Exit\n";

cout << "Enter choice: ";
cin >> choice;

switch(choice) {

   case 1:

      cout << "Enter ID: ";
      cin >> id;

      cout << "Enter Name: ";
      cin >> name;

      db.insertRecord(id, name);

      break;

   case 2:

      cout << "Enter ID to delete: ";
      cin >> id;

      db.deleteRecord(id);
```

```
    break;

case 3:

    cout << "Enter ID to modify: ";
    cin >> id;

    db.modifyRecord(id);

    break;

case 4:

    db.displayForward();

    break;

case 5:

    db.displayBackward();

    break;

case 6:

    cout << "Exiting...\n";

    break;

default:
```

```
            cout << "Invalid Choice\n";
        }

    } while(choice != 6);

    return 0;
}
```

```
#include<iostream>
#include<cctype>
#include<cmath>
using namespace std;

// Node for Stack
struct node{

   int data;
   node* next;
};

// Stack ADT using Linked List
class stackADT{

   node* top;

public:

   stackADT(){
      top = NULL;
   }

   // Push
   void push(int x){

      node* newnode = new node();

      newnode->data = x;
      newnode->next = top;
```

```
    top = newnode;
}

// Pop
int pop(){

    if(top == NULL){
        return -1;
    }

    int val = top->data;

    node* temp = top;

    top = top->next;

    delete temp;

    return val;
}

// Peek
int peek(){

    if(top == NULL){
        return -1;
    }

    return top->data;
}
```

```
    // Empty check
    bool isempty(){

        return top == NULL;
    }
};

// Priority function
int precedence(char op){

    if(op == '^')
        return 3;

    else if(op == '*' || op == '/')
        return 2;

    else if(op == '+' || op == '-')
        return 1;

    return 0;
}

// Infix to Postfix
string infixToPostfix(string infix){

    stackADT s;

    string postfix = "";

    for(int i=0; i<infix.length(); i++){
```

```
char ch = infix[i];

// Operand
if(isalnum(ch)){
   postfix += ch;
}

// Opening bracket
else if(ch == '('){
   s.push(ch);
}

// Closing bracket
else if(ch == ')'){

   while(s.peek() != '('){
      postfix += char(s.pop());
   }

   s.pop();
}

// Operator
else{

   while(!s.isempty() &&
         precedence(ch) <= precedence(s.peek())){

      postfix += char(s.pop());
   }
```

```
            s.push(ch);
        }
    }

    // Remaining operators
    while(!s.isempty()){

        postfix += char(s.pop());
    }

    return postfix;
}

// Reverse string
string reverseString(string str){

    string rev = "";

    for(int i=str.length()-1; i>=0; i--){
        rev += str[i];
    }

    return rev;
}

// Infix to Prefix
string infixToPrefix(string infix){

    // Reverse infix
    infix = reverseString(infix);
```

```
    // Change brackets
    for(int i=0;i<infix.length();i++){

        if(infix[i]=='(')
            infix[i]=')';

        else if(infix[i]==')')
            infix[i]='(';
    }

    // Convert to postfix
    string postfix = infixToPostfix(infix);

    // Reverse postfix
    return reverseString(postfix);
}

// Evaluate Postfix
int evaluatePostfix(string postfix){

    stackADT s;

    for(int i=0;i<postfix.length();i++){

        char ch = postfix[i];

        // Operand
        if(isdigit(ch)){

            s.push(ch - '0');
        }
```

```
    // Operator
    else{

        int b = s.pop();
        int a = s.pop();

        switch(ch){

            case '+':
                s.push(a+b);
                break;

            case '-':
                s.push(a-b);
                break;

            case '*':
                s.push(a*b);
                break;

            case '/':
                s.push(a/b);
                break;

            case '^':
                s.push(pow(a,b));
                break;
        }
    }
}
```

```
    return s.pop();
}

// Evaluate Prefix
int evaluatePrefix(string prefix){

    stackADT s;

    for(int i=prefix.length()-1;i>=0;i--){

        char ch = prefix[i];

        // Operand
        if(isdigit(ch)){

            s.push(ch - '0');
        }

        // Operator
        else{

            int a = s.pop();
            int b = s.pop();

            switch(ch){

                case '+':
                    s.push(a+b);
                    break;
```

```
            case '-':
                s.push(a-b);
                break;

            case '*':
                s.push(a*b);
                break;

            case '/':
                s.push(a/b);
                break;

            case '^':
                s.push(pow(a,b));
                break;
        }
      }
   }

   return s.pop();
}

int main(){

   string infix;

   cout<<"Enter Infix Expression: ";
   cin>>infix;

   string postfix = infixToPostfix(infix);
```

```
    string prefix = infixToPrefix(infix);

    cout<<"\nPostfix Expression = "<<postfix<<endl;

    cout<<"Prefix Expression = "<<prefix<<endl;

    // Evaluation examples
    string postexp = "23*5+";
    string preexp = "+*235";

    cout<<"\nPostfix Evaluation = "
       <<evaluatePostfix(postexp)<<endl;

    cout<<"Prefix Evaluation = "
       <<evaluatePrefix(preexp)<<endl;

    return 0;
}
```

```
#include<iostream>
using namespace std;

class CircularQueue {

   int arr[5];
   int front;
   int rear;
   int size;

public:

   CircularQueue() {

      front = -1;
      rear = -1;
      size = 5;
   }

   // Enqueue
   void enqueue(int value) {

      // Queue Full
      if((rear + 1) % size == front) {

         cout<<"Queue is Full\n";
         return;
      }

      // First element
      if(front == -1) {
```

```
        front = rear = 0;
    }

    else {

        rear = (rear + 1) % size;
    }

    arr[rear] = value;

    cout<<value<<" Inserted\n";
}

// Dequeue
void dequeue() {

    // Queue Empty
    if(front == -1) {

        cout<<"Queue is Empty\n";
        return;
    }

    cout<<arr[front]<<" Deleted\n";

    // Only one element
    if(front == rear) {

        front = rear = -1;
    }
```

```
    else {

        front = (front + 1) % size;
    }
}

// Display
void display() {

    if(front == -1) {

        cout<<"Queue is Empty\n";
        return;
    }

    cout<<"Circular Queue: ";

    int i = front;

    while(i != rear) {

        cout<<arr[i]<<" ";

        i = (i + 1) % size;
    }

    cout<<arr[rear];

    cout<<endl;
}
```

```
};

int main() {

    CircularQueue q;

    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);

    q.display();

    q.dequeue();

    q.display();

    q.enqueue(50);
    q.enqueue(60);

    q.display();

    return 0;
}
```

```
#include<iostream>
#include<queue>
using namespace std;

// Node Structure
struct node{

   int data;
   node* left;
   node* right;
};

// BST Class
class BST{

public:

   node* root;

   BST(){

      root = NULL;
   }

   // Insert Node
   node* insert(node* root,int value){

      if(root == NULL){

         node* newnode = new node();
```

```
        newnode->data = value;
        newnode->left = NULL;
        newnode->right = NULL;

        return newnode;
    }

    if(value < root->data){

        root->left = insert(root->left,value);
    }

    else{

        root->right = insert(root->right,value);
    }

    return root;
}

// Find Minimum Node
node* minValue(node* root){

    while(root->left != NULL){

        root = root->left;
    }

    return root;
}
```

```
// Delete Node
node* deletenode(node* root,int key){

   if(root == NULL){

      return root;
   }

   // Search node
   if(key < root->data){

      root->left = deletenode(root->left,key);
   }

   else if(key > root->data){

      root->right = deletenode(root->right,key);
   }

   else{

      // No child
      if(root->left == NULL &&
         root->right == NULL){

         delete root;

         return NULL;
      }

      // One child
```

```
        else if(root->left == NULL){

            node* temp = root->right;

            delete root;

            return temp;
        }

        else if(root->right == NULL){

            node* temp = root->left;

            delete root;

            return temp;
        }

        // Two children
        node* temp = minValue(root->right);

        root->data = temp->data;

        root->right =
        deletenode(root->right,temp->data);
    }

    return root;
}

// Search
```

```
void search(node* root,int key){

   if(root == NULL){

      cout<<"Element Not Found\n";

      return;
   }

   if(root->data == key){

      cout<<"Element Found\n";

      return;
   }

   if(key < root->data){

      search(root->left,key);
   }

   else{

      search(root->right,key);
   }
}

// Mirror Image
node* mirror(node* root){

   if(root == NULL){
```

```
        return NULL;
    }

    node* temp = root->left;

    root->left = root->right;

    root->right = temp;

    mirror(root->left);

    mirror(root->right);

    return root;
}

// Inorder Display
void display(node* root){

    if(root == NULL){

        return;
    }

    display(root->left);

    cout<<root->data<<" ";

    display(root->right);
}
```

```
// Level Wise Display
void levelwise(node* root){

   if(root == NULL){

      return;
   }

   queue<node*> q;

   q.push(root);

   while(!q.empty()){

      node* temp = q.front();

      q.pop();

      cout<<temp->data<<" ";

      if(temp->left != NULL){

         q.push(temp->left);
      }

      if(temp->right != NULL){

         q.push(temp->right);
      }
   }
```

```
    }
};

int main(){

    BST b;

    // Insert
    b.root = b.insert(b.root,50);
    b.root = b.insert(b.root,30);
    b.root = b.insert(b.root,70);
    b.root = b.insert(b.root,20);
    b.root = b.insert(b.root,40);
    b.root = b.insert(b.root,60);
    b.root = b.insert(b.root,80);

    cout<<"Inorder Display:\n";

    b.display(b.root);

    cout<<endl;

    // Search
    b.search(b.root,40);

    // Delete
    b.root = b.deletenode(b.root,30);

    cout<<"\nAfter Deletion:\n";

    b.display(b.root);
```

```
    cout<<endl;

    // Mirror
    b.root = b.mirror(b.root);

    cout<<"\nMirror Image:\n";

    b.display(b.root);

    cout<<endl;

    // Level Wise
    cout<<"\nLevel Wise Display:\n";

    b.levelwise(b.root);

    return 0;
}
```

```
Inorder Display:
20 30 40 50 60 70 80
Element Found
After Deletion:
20 40 50 60 70 80

Mirror Image:
80 70 60 50 40 20

Level Wise Display:
50 70 40 80 60 20
```

```
#include<iostream>
using namespace std;

#define V 5   // Number of vertices

// Function to find minimum key value
int minkey(int key[], bool visited[]) {

   int min = 999;
   int minindex;

   for(int i=0; i<V; i++) {

      if(visited[i] == false && key[i] < min) {

         min = key[i];
         minindex = i;
      }
   }

   return minindex;
}

// Print MST
void printMST(int parent[], int graph[V][V]) {

   cout << "Edge \tWeight\n";

   for(int i=1; i<V; i++) {

      cout << parent[i]
```

```
            << " - " << i
            << "\t" << graph[i][parent[i]]
            << endl;
    }
}

// Prim's Algorithm
void prims(int graph[V][V]) {

    int parent[V];      // Store MST
    int key[V];         // Minimum weights
    bool visited[V];    // Visited vertices

    // Initialize
    for(int i=0; i<V; i++) {

        key[i] = 999;
        visited[i] = false;
    }

    key[0] = 0;
    parent[0] = -1;

    // MST will have V-1 edges
    for(int count=0; count<V-1; count++) {

        int u = minkey(key, visited);

        visited[u] = true;

        // Update adjacent vertices
```

```
      for(int v=0; v<V; v++) {

        if(graph[u][v] &&
          visited[v] == false &&
          graph[u][v] < key[v]) {

           parent[v] = u;
           key[v] = graph[u][v];
        }
      }
   }

   printMST(parent, graph);
}

int main() {

   // Adjacency Matrix
   int graph[V][V] = {

      {0, 2, 0, 6, 0},
      {2, 0, 3, 8, 5},
      {0, 3, 0, 0, 7},
      {6, 8, 0, 0, 9},
      {0, 5, 7, 9, 0}
   };
   cout << "Minimum Spanning Tree using Prim's Algorithm\n\n";
   prims(graph);

   return 0;
}
```

```
#include<iostream>
using namespace std;

class Player {

public:

   string name;
   int age;
   string country;
   string category;
   int odi;
   int t20;
   float avgscore;
   int wickets;

} p[100];

int n = 0;

// Insert Record
void insertPlayer() {

   cout<<"Enter Number of Players: ";
   cin>>n;

   for(int i=0;i<n;i++) {

      cout<<"\nEnter Player "<<i+1<<" Details\n";

      cout<<"Name: ";
```

```
        cin>>p[i].name;

        cout<<"Age: ";
        cin>>p[i].age;

        cout<<"Country: ";
        cin>>p[i].country;

        cout<<"Category(Batsman/Bowler/WicketKeeper/AllRounder): ";
        cin>>p[i].category;

        cout<<"ODI Played: ";
        cin>>p[i].odi;

        cout<<"T20 Played: ";
        cin>>p[i].t20;

        cout<<"Average Batting Score: ";
        cin>>p[i].avgscore;

        cout<<"Wickets Taken: ";
        cin>>p[i].wickets;
    }
}

// Display All
void display() {

    for(int i=0;i<n;i++) {

        cout<<"\n----------\n";
```

```
        cout<<"Name: "<<p[i].name<<endl;
        cout<<"Age: "<<p[i].age<<endl;
        cout<<"Country: "<<p[i].country<<endl;
        cout<<"Category: "<<p[i].category<<endl;
        cout<<"ODI: "<<p[i].odi<<endl;
        cout<<"T20: "<<p[i].t20<<endl;
        cout<<"Average Score: "<<p[i].avgscore<<endl;
        cout<<"Wickets: "<<p[i].wickets<<endl;
    }
}

// Number of batsmen from country
void batsmanCountry() {

    string c;
    int count = 0;

    cout<<"Enter Country: ";
    cin>>c;

    for(int i=0;i<n;i++) {

        if(p[i].country == c &&
           p[i].category == "Batsman") {

            count++;
        }
    }

    cout<<"Number of Batsmen = "<<count<<endl;
```

```
}

// Sort batsman by average score
void sortBatsman() {

   for(int i=0;i<n-1;i++) {

      for(int j=0;j<n-i-1;j++) {

         if(p[j].avgscore < p[j+1].avgscore) {

            Player temp;

            temp = p[j];
            p[j] = p[j+1];
            p[j+1] = temp;
         }
      }
   }

   cout<<"\nSorted Batsmen:\n";

   for(int i=0;i<n;i++) {

      if(p[i].category == "Batsman") {

         cout<<p[i].name
            <<" -> "
            <<p[i].avgscore
            <<endl;
      }
```

```
    }
}

// Highest average score
void highestAverage() {

    float max = p[0].avgscore;
    int index = 0;

    for(int i=1;i<n;i++) {

        if(p[i].avgscore > max) {

            max = p[i].avgscore;
            index = i;
        }
    }

    cout<<"\nHighest Average Score Player\n";

    cout<<"Name: "<<p[index].name<<endl;
    cout<<"Average: "<<p[index].avgscore<<endl;
}

// Number of bowlers from country
void bowlerCountry() {

    string c;
    int count = 0;

    cout<<"Enter Country: ";
```

```
    cin>>c;

    for(int i=0;i<n;i++) {

        if(p[i].country == c &&
           p[i].category == "Bowler") {

            count++;
        }
    }

    cout<<"Number of Bowlers = "<<count<<endl;
}

// Maximum wickets
void maxWickets() {

    int max = p[0].wickets;
    int index = 0;

    for(int i=1;i<n;i++) {

        if(p[i].wickets > max) {

            max = p[i].wickets;
            index = i;
        }
    }

    cout<<"\nMaximum Wicket Taker\n";
```

```
    cout<<"Name: "<<p[index].name<<endl;
    cout<<"Wickets: "<<p[index].wickets<<endl;
}

// Search Player
void searchPlayer() {

    string name;

    cout<<"Enter Player Name: ";
    cin>>name;

    for(int i=0;i<n;i++) {

        if(p[i].name == name) {

            cout<<"\nPlayer Found\n";

            cout<<"Name: "<<p[i].name<<endl;
            cout<<"Age: "<<p[i].age<<endl;
            cout<<"Country: "<<p[i].country<<endl;
            cout<<"Category: "<<p[i].category<<endl;
            cout<<"ODI: "<<p[i].odi<<endl;
            cout<<"T20: "<<p[i].t20<<endl;
            cout<<"Average: "<<p[i].avgscore<<endl;
            cout<<"Wickets: "<<p[i].wickets<<endl;

            return;
        }
    }
```

```
    cout<<"Player Not Found\n";
}

// Delete Record
void deletePlayer() {

    string name;

    cout<<"Enter Player Name to Delete: ";
    cin>>name;

    for(int i=0;i<n;i++) {

        if(p[i].name == name) {

            for(int j=i;j<n-1;j++) {

                p[j] = p[j+1];
            }

            n--;

            cout<<"Record Deleted\n";

            return;
        }
    }

    cout<<"Player Not Found\n";
}
```

```
// Modify Record
void modifyPlayer() {

   string name;

   cout<<"Enter Player Name to Modify: ";
   cin>>name;

   for(int i=0;i<n;i++) {

      if(p[i].name == name) {

         cout<<"Enter New Age: ";
         cin>>p[i].age;

         cout<<"Enter New ODI: ";
         cin>>p[i].odi;

         cout<<"Enter New T20: ";
         cin>>p[i].t20;

         cout<<"Enter New Average Score: ";
         cin>>p[i].avgscore;

         cout<<"Enter New Wickets: ";
         cin>>p[i].wickets;

         cout<<"Record Modified\n";

         return;
      }
```

```
    }

    cout<<"Player Not Found\n";
}

int main() {

    int choice;

    do {

        cout<<"\n----- MENU -----\n";

        cout<<"1. Insert Player\n";
        cout<<"2. Display All\n";
        cout<<"3. Number of Batsman of Country\n";
        cout<<"4. Sort Batsman by Average\n";
        cout<<"5. Highest Average Score\n";
        cout<<"6. Number of Bowlers of Country\n";
        cout<<"7. Maximum Wickets\n";
        cout<<"8. Search Player\n";
        cout<<"9. Delete Player\n";
        cout<<"10. Modify Player\n";
        cout<<"11. Exit\n";

        cout<<"Enter Choice: ";
        cin>>choice;

        switch(choice) {

            case 1:
```

```
    insertPlayer();
    break;

case 2:
    display();
    break;

case 3:
    batsmanCountry();
    break;

case 4:
    sortBatsman();
    break;

case 5:
    highestAverage();
    break;

case 6:
    bowlerCountry();
    break;

case 7:
    maxWickets();
    break;

case 8:
    searchPlayer();
    break;
```

```
            case 9:
                deletePlayer();
                break;

            case 10:
                modifyPlayer();
                break;

            case 11:
                cout<<"Exiting...\n";
                break;

            default:
                cout<<"Invalid Choice\n";
        }

    } while(choice != 11);

    return 0;
}
```